

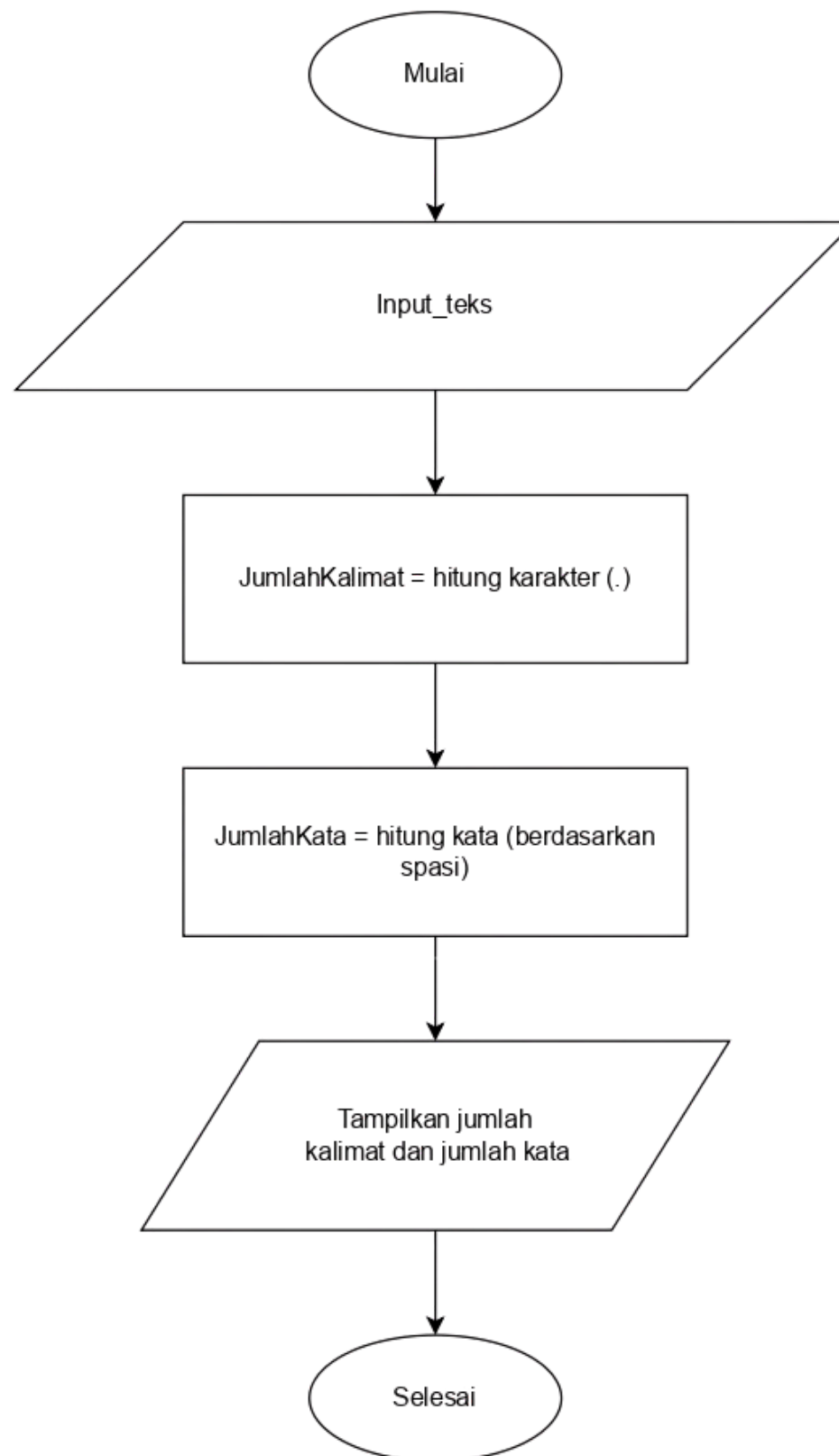


Project **Algoritma Pemrograman**

Disusun oleh Kelompok G:

- Aurn Nathan F. (K1D025016)
- Rizky Aulia P. (K1D025029)
- Lanang Tegar H. (K1D025040)
- Muhammad Aji F. (K1D025041)

PROGRAM 1



PROGRAM 1

Skenario 1 :

Program diuji menggunakan teks yang terdiri atas beberapa kalimat, yaitu tentang aktivitas ibu, ayah, kucing, dan burung. Hasil pengujian menunjukkan bahwa program berhasil menghitung jumlah kalimat berdasarkan tanda titik (.) serta menghitung jumlah kata melalui proses pemisahan teks berdasarkan spasi. Output yang dihasilkan sesuai dengan isi teks yang diberikan sehingga menunjukkan bahwa program mampu bekerja dengan baik pada teks yang relatif panjang.

Skenario 2

Pengujian kedua dilakukan menggunakan teks yang lebih pendek, yaitu "Indonesia merdeka pada tanggal 17 Agustus 1945.". Program berhasil mengidentifikasi satu kalimat dan menghitung jumlah kata secara tepat. Hasil pengujian menunjukkan bahwa program tetap dapat berfungsi dengan baik meskipun input hanya terdiri atas satu kalimat sederhana.

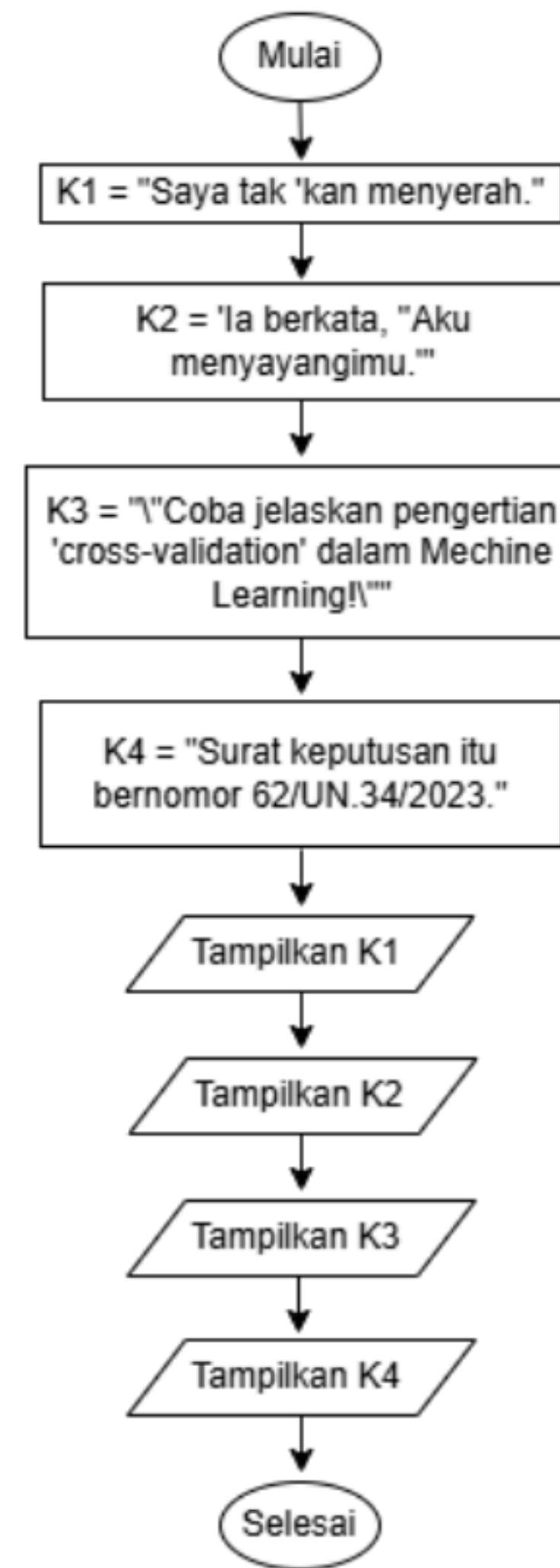
Skenario 3

Pengujian ketiga menggunakan teks yang mengandung nilai numerik, yaitu informasi mengenai harga buku dan jumlah uang yang dibawa. Program berhasil menghitung jumlah kalimat dan jumlah kata tanpa mengalami kesalahan meskipun terdapat angka dalam teks. Hasil ini menunjukkan bahwa keberadaan data numerik tidak memengaruhi proses penghitungan yang dilakukan oleh program.

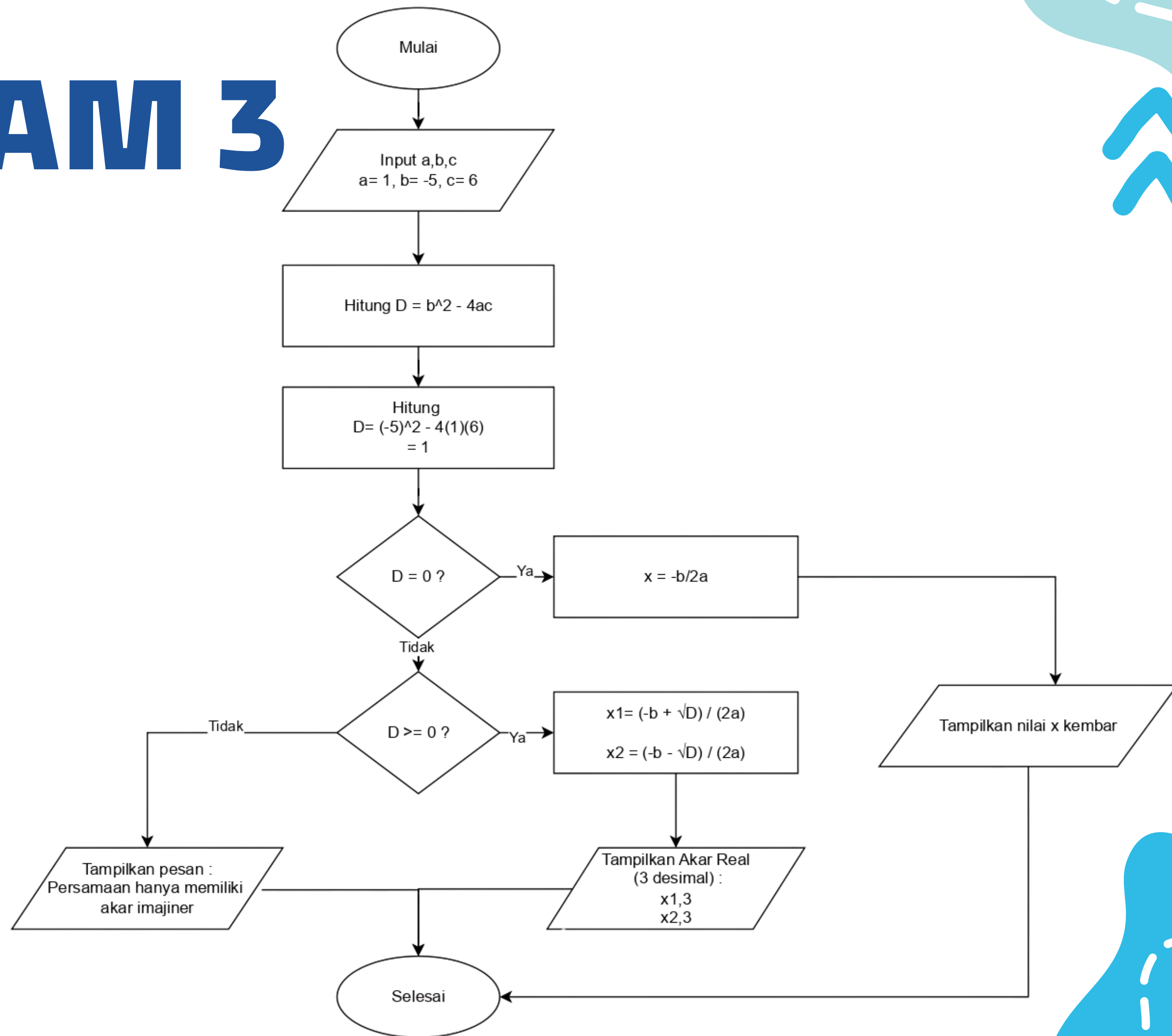
PROGRAM 2

Pengujian Program Tipe Data String (Python & R)

Pengujian ini berfokus pada **penanganan tipe data string yang memuat kombinasi tanda kutip tunggal dan ganda secara bersamaan**. Pada deklarasi variabel ketiga (K3), program menerapkan escape character (\\) agar sistem dapat membedakan antara tanda kutip bagian dari kalimat dan tanda kutip pembungkus kode. Untuk menampilkan output akhir yang bersih tanpa memunculkan karakter backslash, eksekusi pencetakan pada **bahasa Python** menggunakan fungsi standar **print()**, sementara pada bahasa R disesuaikan menggunakan fungsi **cat()**. Hasil pengujian membuktikan bahwa kedua program **berhasil menampilkan** keempat kalimat secara rapi dan identik sesuai dengan teks yang diminta pada ketentuan soal.



PROGRAM 3



PROGRAM 3

Skenario 1

Program diuji menggunakan koefisien $a=1$, $b=2$, dan $c=1$. Nilai diskriminan yang diperoleh adalah $D=0$, sehingga persamaan kuadrat memiliki dua akar yang sama (akar kembar). Program berhasil mengidentifikasi kondisi tersebut dan menghasilkan nilai akar kembar $x=-1$. Hasil yang diperoleh sesuai dengan teori persamaan kuadrat.

Skenario 3

Pada pengujian ketiga digunakan koefisien $a=1$, $b=2$, dan $c=5$. Nilai diskriminan yang diperoleh adalah $=-16$, sehingga persamaan tidak memiliki akar real dan hanya memiliki akar imajiner. Program berhasil mendeteksi kondisi diskriminan negatif dan menampilkan pesan bahwa persamaan hanya memiliki akar imajiner. Hal ini menunjukkan bahwa program dapat membedakan jenis akar berdasarkan nilai diskriminannya.

Skenario 2

Pengujian kedua dilakukan dengan koefisien $a=1$, $b=-5$, dan $c=6$. Nilai diskriminan yang diperoleh adalah $D=1$, sehingga persamaan memiliki dua akar real yang berbeda. Program berhasil menghitung kedua akar tersebut, yaitu $x_1=3$ dan $x_2=2$. Hasil ini menunjukkan bahwa program mampu menangani kasus persamaan kuadrat dengan dua solusi real.

PROGRAM 4

PROGRAM NIP_ASN

DEKLARASI

```
NIP      : STRING
tahun    : INTEGER
bulan    : INTEGER
tanggal  : INTEGER
nama_bulan : STRING
max_hari : INTEGER
```

```
data_bulan : ARRAY
data_hari  : ARRAY
```

ALGORITMA

```
data_bulan ← [1:"Januari", 2:"Februari",
3:"Maret",
4:"April", 5:"Mei", 6:"Juni",
7:"Juli", 8:"Agustus", 9:"September",
10:"Oktober", 11:"November",
12:"Desember"]
```

```
data_hari ← [1:31, 2:28, 3:31, 4:30, 5:31, 6:30,
7:31, 8:31, 9:30, 10:31, 11:30, 12:31]
```

```
LABEL_INPUT:
READ(NIP)
```

```
tahun ← INT(NIP[1:4])
bulan ← INT(NIP[5:6])
tanggal ← INT(NIP[7:8])
```

```
IF (bulan < 1) OR (bulan > 12) THEN
    WRITE("NIP tidak valid, harap masukkan ulang.")
    GOTO LABEL_INPUT
ENDIF
```

```
nama_bulan ← data_bulan[bulan]
max_hari ← data_hari[bulan]
```

```
IF (bulan = 2) AND
    (tahun MOD 400 = 0 OR
    (tahun MOD 4 = 0 AND tahun MOD 100 ≠ 0))
THEN
    max_hari ← 29
ENDIF
```

```
IF (tanggal < 1) OR (tanggal > max_hari) THEN
    WRITE("NIP tidak valid, harap masukkan ulang.")
    GOTO LABEL_INPUT
ENDIF
```

```
WRITE("Tanggal lahir Anda: ",
    tanggal, " ",
    nama_bulan, " ",
    tahun)
```

```
END PROGRAM
```

PROGRAM 4

Skenario 1: NIP dengan Validasi Normal

Pengujian pertama menggunakan input NIP **199410212019031010** untuk menguji kondisi normal. Program berhasil mengekstrak **tahun 1994, bulan 10 (Oktober), dan tanggal 21**. Karena nilai tanggal 21 berada di bawah batas maksimal bulan Oktober (31 hari), data dinyatakan valid. Program langsung menghentikan perulangan dan sukses mencetak output: "**Tanggal lahir Anda: 21 Oktober 1994**".

Skenario 2: NIP dengan Validasi Tahun Kabisat

Pengujian kedua menguji ketepatan logika tahun kabisat melalui input NIP **200002292022011002**. Program mendeteksi bahwa tahun **2000** habis dibagi 400, sehingga batas maksimal bulan Februari otomatis diperbarui menjadi **29 hari**. Hasilnya, input tanggal 29 dinyatakan sah dan program berhasil mencetak output: "**Tanggal lahir Anda: 29 Februari 2000**".

Skenario 3: NIP Tidak Valid (Bukan Kabisat)

Pengujian ketiga menguji penolakan data yang tidak logis melalui input NIP **200302292024011004**. Karena **2003** bukan tahun kabisat, input **tanggal 29** langsung ditolak sebab melebihi batas maksimal 28 hari, lalu program meminta input ulang. Setelah pengguna memasukkan koreksi berupa NIP dengan **tanggal 28** (200302282024011004), data lolos validasi dan mencetak: "**Tanggal lahir Anda: 28 Februari 2003**".

PROGRAM 5

PROGRAM KlasifikasiCluster3D

DEKLARASI

x1, x2, x3 : REAL
dA, dB, dC : REAL

ALGORITMA

WRITE("=== Sistem Klasifikasi Cluster 3D ===")
WRITE("Titik Pusat Cluster Tetap:")
WRITE("A(2,1,3), B(1,-4,6), C(-2,3,-2)")

WRITE("Masukkan koordinat U (x1, x2, x3):")
READ(x1, x2, x3)

IF x1 BUKAN_ANGKA OR
x2 BUKAN_ANGKA OR
x3 BUKAN_ANGKA THEN

WRITE("Tampilkan Salah Input")

ELSE

$dA \leftarrow \text{SQRT}((x1 - 2)^2 + (x2 - 1)^2 + (x3 - 3)^2)$
 $dB \leftarrow \text{SQRT}((x1 - 1)^2 + (x2 - (-4))^2 + (x3 - 6)^2)$
 $dC \leftarrow \text{SQRT}((x1 - (-2))^2 + (x2 - 3)^2 + (x3 - (-2))^2)$

IF (dA = dB) < dC OR (dB = dC) < dA OR (dA = dC) < dB THEN

WRITE("Tampilkan Kondisi Khusus / Nilai Jarak Sama")

ELSE IF (dA < dB AND dB < dC) OR (dA < dC AND dC < dB) THEN

WRITE("Tampilkan Cluster A dan Nilai Jarak")

ELSE IF (dB < dC AND dC < dA) OR (dB < dA AND dA < dC) THEN

WRITE("Tampilkan Cluster B dan Nilai Jarak")

ELSE

WRITE("Tampilkan Cluster C dan Nilai Jarak")

ENDIF

ENDIF

END PROGRAM

PROGRAM 5

Skenario 1: Pengujian Kondisi Khusus (Jarak Sama dan Terdekat)

Pengujian pertama bertujuan untuk memvalidasi kemampuan algoritma dalam mendeteksi titik uji yang memiliki jarak simetris terhadap dua atau lebih pusat cluster, di mana jarak tersebut merupakan nilai minimum. Berdasarkan hasil eksekusi dengan input koordinat $U=(0, 2, 0.5)$, program menghasilkan nilai jarak Euclidean sebesar 3.3541 ke Cluster A, 8.2006 ke Cluster B, dan 3.3541 ke Cluster C. Melalui evaluasi relasional, sistem mengidentifikasi bahwa jarak ke Cluster A dan C bernilai sama serta berstatus sebagai jarak terpendek. Oleh karena itu, program berhasil mengeluarkan output yang menyatakan bahwa titik uji berada pada Kondisi Khusus antara Cluster A dan Cluster C.

Skenario 2: Pengujian Kondisi Normal (Satu Cluster Terdekat)

Pengujian kedua dilakukan untuk memastikan ketepatan program dalam menentukan satu cluster tunggal yang paling dominan tanpa adanya kesamaan jarak. Melalui input koordinat $U=(-1, 2, -1)$, program melakukan kalkulasi jarak spasial dan menghasilkan data numerik yang bervariasi, yaitu 5.0990 untuk Cluster A, 9.4340 untuk Cluster B, dan 1.7321 untuk Cluster C. Struktur logika penentu keputusan berhasil mengidentifikasi secara absolut bahwa nilai pada Cluster C merupakan representasi jarak paling minimum. Konsekuensinya, sistem melewati seluruh tahapan kondisi khusus dan langsung menetapkan hasil klasifikasi sebagai Kondisi Normal untuk Cluster C.

Skenario 3: Pengujian Kondisi Error (Validasi Input Bukan Angka)

Pengujian ketiga berfungsi untuk menguji keandalan sistem (error handling) dalam mengantisipasi kesalahan tipe data input guna mencegah terjadinya kegagalan sistem (system crash). Pengujian dilakukan dengan memasukkan argumen non-numerik, yaitu ("Lanang", 3, 5). Sebelum berpindah ke tahap komputasi geometris, fungsi penapis (filter) pada baris awal mendeteksi adanya tipe data string. Secara otomatis, sistem menghentikan proses kalkulasi jarak demi keamanan data dan langsung menampilkan pesan logikal yang menyatakan terjadi Kondisi Error karena input wajib berupa angka.

PROGRAM 6

Algoritma Menghitung Interval Konfidensi

DEKLARASI

p_hat, alpha, z, margin_of_error : REAL

batas_bawah, batas_atas : REAL

n : INTEGER

BEGIN

Input p_hat, alpha, n

//Validasi input proporsi

IF (p_hat < 0 OR p_hat > 1) THEN

PRINT("Input harus bernilai antara 0 dan 1")

EXIT

ENDIF

//Menentukan nilai z berdasarkan alpha

IF(alpha == 0.1) THEN

z <- 1.645

ELSE IF (alpha == 0.05) THEN

z <- 1.96

ELSE

PRINT("Nilai alpha hanya mendukung 0.05 dan 0.1")

EXIT

ENDIF

//Pehitungan

margin_of_error <- z * SQRT((p_hat * (1-p_hat))/n)

batas_bawah <- p_hat - margin_of_error

batas_atas <- p_hat + margin_of_error

//Output

Print("Interval konfidensi :", batas_bawah, "< p <", batas_atas)

END PROGRAM

PROGRAM 6

Skenario 1 : Input normal

Pengujian dilakukan untuk memastikan keandalan serta akurasi logika program dalam menghitung interval konfidensi proporsi populasi. Pada skenario pertama, dengan memasukkan parameter $p_{\text{hat}} = 0.5$, $\alpha = 0.05$, dan $n = 50$, program beroperasi dengan tepat dengan menetapkan nilai z sebesar 1,96. Hasil kalkulasi menghasilkan margin of error sebesar 0,1386, sehingga diperoleh rentang interval konfidensi antara 0,3614 hingga 0,6386. Pengujian ini mengonfirmasi bahwa program mampu melakukan perhitungan aritmetika secara akurat sesuai dengan rumus statistik yang ditetapkan.

Skenario 2 : Input normal dengan nilai alpha yang berbeda

Pada skenario kedua, pengujian dilakukan dengan parameter $p_{\text{hat}} = 0.5$, $\alpha = 0.1$, dan $n = 50$. Dalam kondisi ini, program secara otomatis menyesuaikan nilai z menjadi 1,645 sesuai dengan tingkat signifikansi yang diberikan. Dengan tingkat kepercayaan yang lebih rendah, program menghasilkan margin of error yang lebih kecil, yaitu 0,1163, yang menghasilkan rentang interval antara 0,3837 hingga 0,6163. Hasil ini menunjukkan bahwa fungsi pemilihan nilai z berdasarkan parameter α telah berjalan sebagaimana mestinya.

Skenario 3 : Input diluar batas ($p_{\text{hat}} > 1$ atau $p_{\text{hat}} < 0$)

Pada skenario ketiga, dilakukan pengujian dengan memasukkan nilai p_{hat} yang berada di luar rentang yang diizinkan, yaitu $p_{\text{hat}} < 0$ atau $p_{\text{hat}} > 1$. Program berhasil menjalankan fungsi validasi data dengan menghentikan eksekusi rumus sebelum perhitungan dimulai. Sistem kemudian menampilkan pesan kesalahan yang informatif kepada pengguna, yang menyatakan bahwa nilai proporsi harus berada dalam rentang 0 hingga 1. Hal ini membuktikan bahwa mekanisme penanganan kesalahan (error handling) telah diimplementasikan dengan baik, sehingga mencegah program mengalami kegagalan sistem (crash) akibat input yang tidak valid.

LINK GITHUB

<https://github.com/lanangharsono/Tugas-Proyek-Alpro-Kelompok-G.git>

The background features a variety of abstract shapes in dark blue and teal. These include solid organic forms, dashed outlines of similar shapes, and a circular shape with a dashed border at the top center. The overall style is modern and minimalist.

**Terima
Kasih**